



Django

Capítulo 9 Trabalhando com Forms

Django

Capítulo 9 - Trabalhando com Forms

Neste capítulo, veremos como criar um formulário no Django que refletirá a estrutura das models. Como exemplo será feita a interface de edição de usuário. Inicialmente será criado um formulário com os dados da tabela de usuário. Depois incluiremos na mesma interface campos da tabela de User. Crie na pasta do projeto uma pasta chamada forms e, dentro dela, crie um arquivo chamado UserUsuarioForm.py e coloque o código abaixo no loja/forms/**UserUsuarioForm.py**:

```
from django.forms import ModelForm
from django import forms
from loja.models.Usuario import Usuario
class UserUsuarioForm(ModelForm):
    class Meta:
        model = Usuario
        fields = ['user', 'perfil', 'aniversario']
        # Usamos '__all__' para exibir todos os campos do formulário
        # fields = '__all__'
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/forms/**UserUsuarioForm.py**

```
# Usamos uma lista para exibir campos específicos
# fields = ['pub_date', 'headline', 'content', 'reporter']
# Usamos exclude para excluir campos específicos do sistema
# exclude = ['']
widgets = {
    'user': forms.HiddenInput(),
    'perfil': forms.Select(attrs={'class': "form-control"}),
    'aniversario': forms.DateInput(attrs={'class':
"form-control", "type": "date"})
}
```

```
loja > forms > UserUsuarioForm.py > ...
1  from django.forms import ModelForm
2  from django import forms
3  from loja.models.Usuario import Usuario
4  class UserUsuarioForm(ModelForm):
5      class Meta:
6          model = Usuario
7          fields = ['user', 'perfil', 'aniversario']
8          # Usamos '__all__' para exibir todos os campos do formulário
9          fields = '__all__'
10         # Usamos uma lista para exibir campos específicos
11         fields = ['pub_date', 'headline', 'content', 'reporter']
12         # Usamos exclude para excluir campos específicos do sistema
13         exclude = ['']
14         widgets = {'user': forms.HiddenInput(), 'perfil': forms.Select(attrs={'class': "form-control"}),
15                   'aniversario': forms.DateInput(attrs={'class': "form-control", "type": "date"})}
```

Django

Capítulo 9 - Trabalhando com Forms

Para prevenir que um usuário que não seja administrador possa alterar o campo perfil de um usuário pode-se definir uma rotina de inicialização do formulário. Inclua o código a destacado abaixo antes da classe meta, para exibir o campo perfil do formulário.

```
from django.forms import ModelForm
from django import forms
from loja.models.Usuario import Usuario
# Adicione esse trecho antes da classe meta
class UserUsuarioForm(ModelForm):
    def __init__(self, *args, **kwargs):
        super(UserUsuarioForm, self).__init__(*args, **kwargs)
        if self.instance and self.instance.perfil != 1:
            del self.fields['perfil'] #deleta campo se o usuário não
for adm
# Adicione esse trecho antes da classe meta
class Meta:
    #resto do código
```

Django

Capítulo 9 - Trabalhando com Forms

Abra o arquivo **UsuarioView.py** e adicione uma def chamada `edit_usuario`. Nela, vamos adicionar a lógica que vai integrar com o formulário para exibi-lo no template html.

Adicione os trechos destacados abaixo.

Adicione o import do método `get_object_or_404` abaixo

```
from django.shortcuts import render, redirect, get_object_or_404
```

```
from loja.models import Usuario
```

Importe a classe `UserUsuarioForm` abaixo

```
from loja.forms.UserUsuarioForm import UserUsuarioForm
```

```
def list_usuario_view(request, id=None):
```

```
    # carrega somente usuarios, não inclui os admin
```

```
    usuarios = Usuario.objects.filter(perfil=2)
```

```
    context = {
```

```
        'usuarios': usuarios
```

```
    }
```

```
    return render(request, template_name='usuario/usuario.html', context=context,  
status=200)
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do conteúdo do loja/views/**UsuarioView.py**

adicione o método de edição

```
def edit_usuario_view(request):
```

```
    usuario = get_object_or_404(Usuario, user=request.user)
```

```
    usuarioForm = UserUsuarioForm(instance=usuario)
```

```
    context = {
```

```
        'usuarioForm': usuarioForm
```

```
    }
```

```
    return render(request, template_name='usuario/usuario-edit.html', context=context,  
status=200)
```

Django

Capítulo 9 - Trabalhando com Forms

loja/views/**UsuarioView.py**

loja > views >  UsuarioView.py > ...

```
1  # Adicione o import do método get_object_or_404 abaixo
2  from django.shortcuts import render, redirect, get_object_or_404
3  from loja.models import Usuario
4  # importe a classe UserUsuarioForm abaixo
5  from loja.forms.UserUsuarioForm import UserUsuarioForm
6  def list_usuario_view(request, id=None):
7      # carrega somente usuarios, não inclui os admin
8      usuarios = Usuario.objects.filter(perfil=2)
9      context = {
10         'usuarios': usuarios
11     }
12     return render(request, template_name='usuario/usuario.html', context=context, status=200)
13 # adicione o metodo de edição
14 def edit_usuario_view(request):
15     usuario = get_object_or_404(Usuario, user=request.user)
16     usuarioForm = UserUsuarioForm(instance=usuario)
17     context = {
18         'usuarioForm': usuarioForm
19     }
20     return render(request, template name='usuario/usuario-edit.html', context=context, status=200)
```


Django

Capítulo 9 - Trabalhando com Forms

Para criar a Url da visão, altere o arquivo usuarioUrls.py e inclua a rota de edição conforme mostrado a seguir nos trechos destacados.

```
from django.urls import path
from loja.views.UsuarioView import list_usuario_view, edit_usuario_view
urlpatterns = [
    path("", list_usuario_view, name='usuarios'),
    # Adicione a linha a seguir
    path("edit", edit_usuario_view, name='edit_usuario'),
]
```



```
loja > urls > UsuarioUrls.py > ...
1  from django.urls import path
2  from loja.views.UsuarioView import list_usuario_view, edit_usuario_view
3  urlpatterns = [
4      path("", list_usuario_view, name='usuario'),
5      # Adicione a linha a seguir
6      path("edit", edit_usuario_view, name='edit_usuario'),
7  ]
```


Django

Capítulo 9 - Trabalhando com Forms

Será criado agora um template html que utiliza o form. Crie o arquivo **usuario-edit.html** dentro da pasta templates/usuario. Coloque o código abaixo no templates/usuario/usuario-edit.html

```
{% extends "base.html" %}
{% load static %}
{% block title %}Usuário{% endblock %}
{% block styles %}<link rel="stylesheet" href="{% static 'css/home.css' %}">{% endblock %}
{% block content %}
<div id="content">
<div class="container-fluid">
<div class="row">
<form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4" method="POST" action="">
    <div class="row">
        {% csrf_token %}
        {% for f in usuarioForm %}
```

Django

Capítulo 9 - Trabalhando com Forms

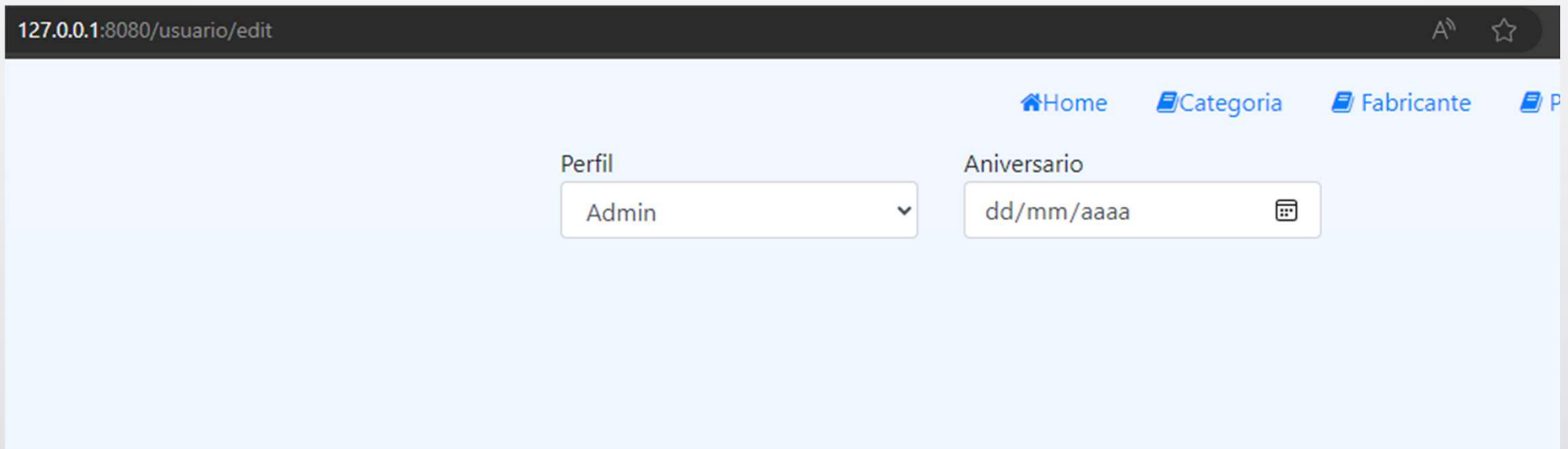
Parte 2 do conteudo loja/templates/usuario/usuario-edit.h

```
{% if not f.is_hidden %}
<div class="form-group col-md-6">
  {{ f.label }}
  {{ f }}
</div>
{% else %}
  {{ f }}
{% endif %}
{% endfor %}
</div>
</form>
</div>
</div>
</div>
{% endblock %}
```

Django

Capítulo 9 - Trabalhando com Forms

Execute a aplicação e acesse o endereço: `http://127.0.0.1:8080/usuario/edit`



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8080/usuario/edit`. The page has a light blue background. At the top right, there are navigation links: [Home](#), [Categoria](#), [Fabricante](#), and [P](#). Below these links, there are two form fields. The first field is labeled "Perfil" and contains a dropdown menu with the text "Admin" and a downward arrow. The second field is labeled "Aniversario" and contains the text "dd/mm/aaaa" followed by a calendar icon.

OBS: Para acessar essa página é necessário ter o perfil do admin criado na nossa tabela de usuários e estar logado na conta dele.

Django

Capítulo 9 - Trabalhando com Forms

Explicando:

A variável **f** recebe a iteração do objeto **form**, que possui uma lista com todos os campos do objeto. Com isso acessamos cada campo do formulário.

A tag `{{f}}` inclui no html o componente visual input, select ou textarea referente aquele campo de acordo com o tipo do mesmo.

`{{f.label}}` retorna o nome do rótulo do campo.

`f.is_hidden` retorna se o campo é do tipo escondido.

É possível acessar os campo do **form** pelo nome dos campos.

Django

Capítulo 9 - Trabalhando com Forms

É possível acessar os campo do **form** pelo nome dos campos. O código abaixo também é válido.

Não incluir esse código, ele gera a mesma apresentação do anterior

```
<form class="col-md-6 col-lg-6 offset-md-3 offset-lg-3" method="POST" action="">
  <div class="row">
    {% csrf_token %}
    <div class="form-group">
      {{ usuarioForm.perfil.label }}
      {{ usuarioForm.perfil }}
    </div>
    <div class="form-group">
      {{ usuarioForm.aniversario.label }}
      {{ usuarioForm.aniversario }}
    </div>
  </div>
</form>
```

Django

Capítulo 9 - Trabalhando com Forms

Será incluído agora os campos referente ao cadastro de usuários que estão armazenados na tabela nativa de usuários do Django (tabela auth_user). Serão incluídos na interface os campos username, email, first_name e last_name. Abra o arquivo **UserUsuarioForm.py** dentro da pasta loja/forms e adicione a classe abaixo.

```
from django.forms import ModelForm
from django import forms
from loja.models.Usuario import Usuario
# Adicione a linha a seguir no import
from django.contrib.auth.models import User
class UserUsuarioForm(ModelForm):
    #Resto do codigo
# Adicione o código da classe UserForm
class UserForm(ModelForm):
    class Meta:
        model = User
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/forms/**UserUsuarioForm.py**:

```
fields = ['username', 'email', 'first_name', 'last_name']
widgets = {
    'username': forms.TextInput(attrs={'class': "form-control"}),
    'email': forms.EmailInput(attrs={'class': "form-control"}),
    'first_name': forms.TextInput(attrs={'class': "form-control"}),
    'last_name': forms.TextInput(attrs={'class': "form-control"})
}
```

```
loja > forms > UserUsuarioForm.py > ...
1  from django.forms import ModelForm
2  from django import forms
3  from loja.models.Usuario import Usuario
4  # Adicione a linha a seguir no import
5  from django.contrib.auth.models import User
6  > class UserUsuarioForm(ModelForm): ...
25 class UserForm(ModelForm):
26     class Meta:
27         model = User
28         fields = ['username', 'email', 'first_name', 'last_name']
29         widgets = {
30             'username': forms.TextInput(attrs={'class': "form-control"}),
31             'email': forms.EmailInput(attrs={'class': "form-control"}),
32             'first_name': forms.TextInput(attrs={'class': "form-control"}),
33             'last_name': forms.TextInput(attrs={'class': "form-control"})
34         }
```


Django

Capítulo 9 - Trabalhando com Forms

O formulário para editar as partes do usuário que ficam na tabela padrão do Django foi definido. Agora é necessário adicionar esses campos na visão e no template. Na pasta loja/views acesse o arquivo UsuarioView.py e faça as alterações destacadas abaixo:

```
from django.shortcuts import render, redirect, get_object_or_404
from loja.models import Usuario
# Adicione a classe UserForm
from loja.forms.UserUsuarioForm import UserUsuarioForm, UserForm
def list_usuario_view(request, id=None):
    #Resto do codigo
# Altere o metodo de edição
def edit_usuario_view(request):
    usuario = get_object_or_404(Usuario, user=request.user)
    usuarioForm = UserUsuarioForm(instance=usuario)
    # Crie uma instância da classe UserForm
    userForm = UserForm(instance=request.user)
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/views/**UsuarioView.py**:

```
# Adicione uma chave userForm no contexto
context = {
    'usuarioForm': usuarioForm,
    'userForm': userForm
}

return render(request, template_name='usuario/usuario-edit.html',
context=context, status=200)
```

```
loja > views > UsuarioView.py > ...
1 # Adicione o import do método get_object_or_404 abaixo
2 from django.shortcuts import render, redirect, get_object_or_404
3 from loja.models import Usuario
4 # Adicione a classe UserForm
5 from loja.forms.UserUsuarioForm import UserUsuarioForm, UserForm
6 > def list_usuario_view(request, id=None): ...
13 # adicione o método de edição
14 def edit_usuario_view(request):
15     print(request.user)
16     usuario = get_object_or_404(Usuario, user=request.user)
17     usuarioForm = UserUsuarioForm(instance=usuario)
18     # Crie uma instância da classe UserForm
19     userForm = UserForm(instance=request.user)
20     # Adicione uma chave userForm no contexto
21     context = {
22         'usuarioForm': usuarioForm,
23         'userForm': userForm
24     }
25     return render(request, template_name='usuario/usuario-edit.html', context=context, status=200)
```

Django

Capítulo 9 - Trabalhando com Forms

Abra o arquivo **usuario-edit.html** dentro da pasta loja/templates/usuario e inclua o código a seguir com os componentes visuais dos novos campos

```
{% extends "base.html" %}
<!--Resto do código-->
<form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4" method="POST" action="">
    <div class="row">
        {% csrf_token %}
        <!-- Adicione o userForm antes do usuarioForm -->
        {% for f in userForm %}
            {% if not f.is_hidden %}
                <div class="form-group col-md-6">
                    {{ f.label }}
                    {{ f }}
                </div>
            {% else %}
                {{ f }}
            {% endif %}
        {% endfor %}
    </div>
</form>
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/templates/usuario/**usuario-edit.html**:

```
        {% endif %}
    {% endfor %}
    <!-- Até aqui -->
    {% for f in usuarioForm %}
        {% if not f.is_hidden %}
        <div class="form-group col-md-6">
            {{ f.label }}
            {{ f }}
        </div>
        {% else %}
            {{ f }}
        {% endif %}
    {% endfor %}
</div>
</form></div></div></div>{% endblock %}
```

Django

Capítulo 9 - Trabalhando com Forms

Imagem do código do loja/templates/usuario/**usuario-edit.html**:

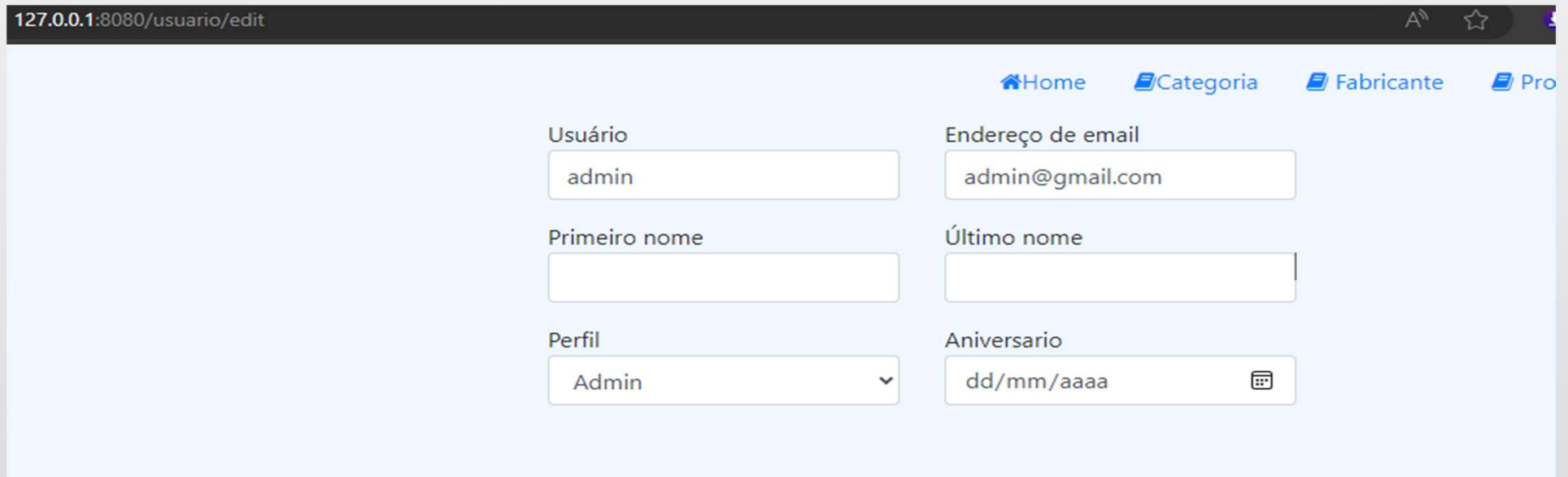
```
loja > templates > usuario > <> usuario-edit.html > ...
1  {% extends 'base.html' %}
2  {% load static %}
3  {% block title %}Usuário{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">{% endblock %}
6  {% block content %}
7  <div id="content">
8      <div class="container-fluid">
9          <div class="row">
10             <form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4" method="POST" action="">
11                 <div class="row">
12                     {% csrf_token %}
13                     <!-- Adicione o userForm antes do usuarioForm -->
14                     {% for f in userForm %}
15                     {% if not f.is_hidden %}
16                     <div class="form-group col-md-6">
17                         {{ f.label }}
18                         {{ f }}
19                     </div>
20                     {% else %}
21                     {{ f }}
22                     {% endif %}
23                     {% endfor %}
24                     <!-- Até aqui -->
25                     {% for f in usuarioForm %}
26                     {% if not f.is_hidden %}
27                     <div class="form-group col-md-6">
28                         {{ f.label }}
29                         {{ f }}
30                     </div>
31                     {% else %}
32                     {{ f }}
33                     {% endif %}
34                     {% endfor %}
35                 </div>
36             </form>
37         </div>
38     </div>
39 </div>
40 {% endblock %}
```

Django

Capítulo 9 - Trabalhando com Forms

A apresentação da tela de edição agora está conforme apresentado abaixo. Para acessá-la use o endereço `http://127.0.0.1:8080/usuario/edit`

OBS: Para acessar a página é necessário ter realizado o acesso ao endereço de administração (`http://127.0.0.1:8080/admin`) e realizar o login.



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:8080/usuario/edit`. The page has a light blue background and a navigation bar at the top with links: Home, Categoria, Fabricante, and Pro. The main content area contains a form with the following fields:

Usuário	Endereço de email
admin	admin@gmail.com
Primeiro nome	Último nome
Perfil	Aniversario
Admin	dd/mm/aaaa

Django

Capítulo 9 - Trabalhando com Forms

A interface de edição de perfil pode ser vista mas ainda não está com a funcionalidade implementada. É preciso fazer ainda uma mensagem de validação e de sucesso ou erro do salvamento, botão de enviar formulário e gravar os dados no banco de dados. Para começar, Inclua o código html do botão de salvar no arquivo `loja/templates/usuario/usuario-edit.html` conforme apresentado a seguir:

```
{% extends "base.html" %}
<!--Resto do código-->
<form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4"
    method="POST" action="">
    <div class="row">
        <!--Resto do código-->
    </div>
    <!-- Adicione antes do fechamento do form-->
    <button type="submit" class="btn btn-primary">Salvar</button>
</form></div></div></div>{% endblock %}
```

Django

Capítulo 9 - Trabalhando com Forms

E na classe de visão, loja/views/UsuarioView.py, inclua o código abaixo para tratar os dados recebidos no POST e salvar no banco dados.

```
from django.shortcuts import render, redirect, get_object_or_404
from loja.models import Usuario
from loja.forms.UserUsuarioForm import UserUsuarioForm, UserForm
#Resto do código
def edit_usuario_view(request):
    usuario = get_object_or_404(Usuario, user=request.user)
    # Adicione as linhas a seguir
    # Perceba que os forms foram transferidos para dentro do if e do else
    emailUnused = True
    if request.method == 'POST':
        usuarioForm = UserUsuarioForm(request.POST, instance=usuario)
        userForm = UserForm(request.POST, instance=request.user)
        # Verifica se o e-mail que o usuário está tentando utilizar
        # em seu perfil já existe em outro perfil
```


Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/views/UsuarioView.py

```
        verifyEmail =
Usuario.objects.filter(user__email=request.POST['email']).exclude(user__id=request.user.
id).first()
        emailUnused = verifyEmail is None
    else:
        usuarioForm = UserUsuarioForm(instance=usuario)
        userForm = UserForm(instance=request.user)
        if usuarioForm.is_valid() and userForm.is_valid() and emailUnused:
            usuarioForm.save()
            userForm.save()
        # Até aqui antes do context
        context = {
            'usuarioForm': usuarioForm,
            'userForm': userForm
        }
        return render(request, template_name='usuario/usuario-edit.html', context=context, status=200
```

Django

Capítulo 9 - Trabalhando com Forms

Agora será adicionado os campos de mensagem de validação no formulário de modo que ele informe ao usuário quando um campo está com formato inválido. O objeto `errors` dentro de cada item do formulário, que retorna uma lista que contém um ou mais erros de validação. Inclua os comandos a seguir no template, `loja/templates/usuario/usuario-edit.html`, para incluir os componentes de validação

```
{% extends "base.html" %}
<!--Resto do código-->
    {% for f in userForm %}
        {% if not f.is_hidden %}
            <div class="form-group col-md-6">
                {{ f.label }}
                {{ f }}
                <!-- Adicione as linhas a seguir -->
                {% for error in f.errors %}
                    <div class="invalid-feedback" style="display: initial;">{{ error }}</div>
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do loja/templates/usuario/**usuario-edit.html**:

```
        {% endfor %}
        <!-- Até aqui -->
    </div>
<!--Resto do código-->
    {% for f in usuarioForm %}
        {% if not f.is_hidden %}
            <div class="form-group col-md-6">
                {{ f.label }}
                {{ f }}
                <!-- Adicione a linha a seguir -->
                {% for error in f.errors %}
                    <div class="invalid-feedback" style="display: initial;">{{ error }}</div>
                {% endfor %}
                <!-- Até aqui -->
            </div>{% else %}{{ f }}{% endif %}{% endfor %}<!--Resto do código-->
```

Capítulo 9 - Trabalhando com Forms

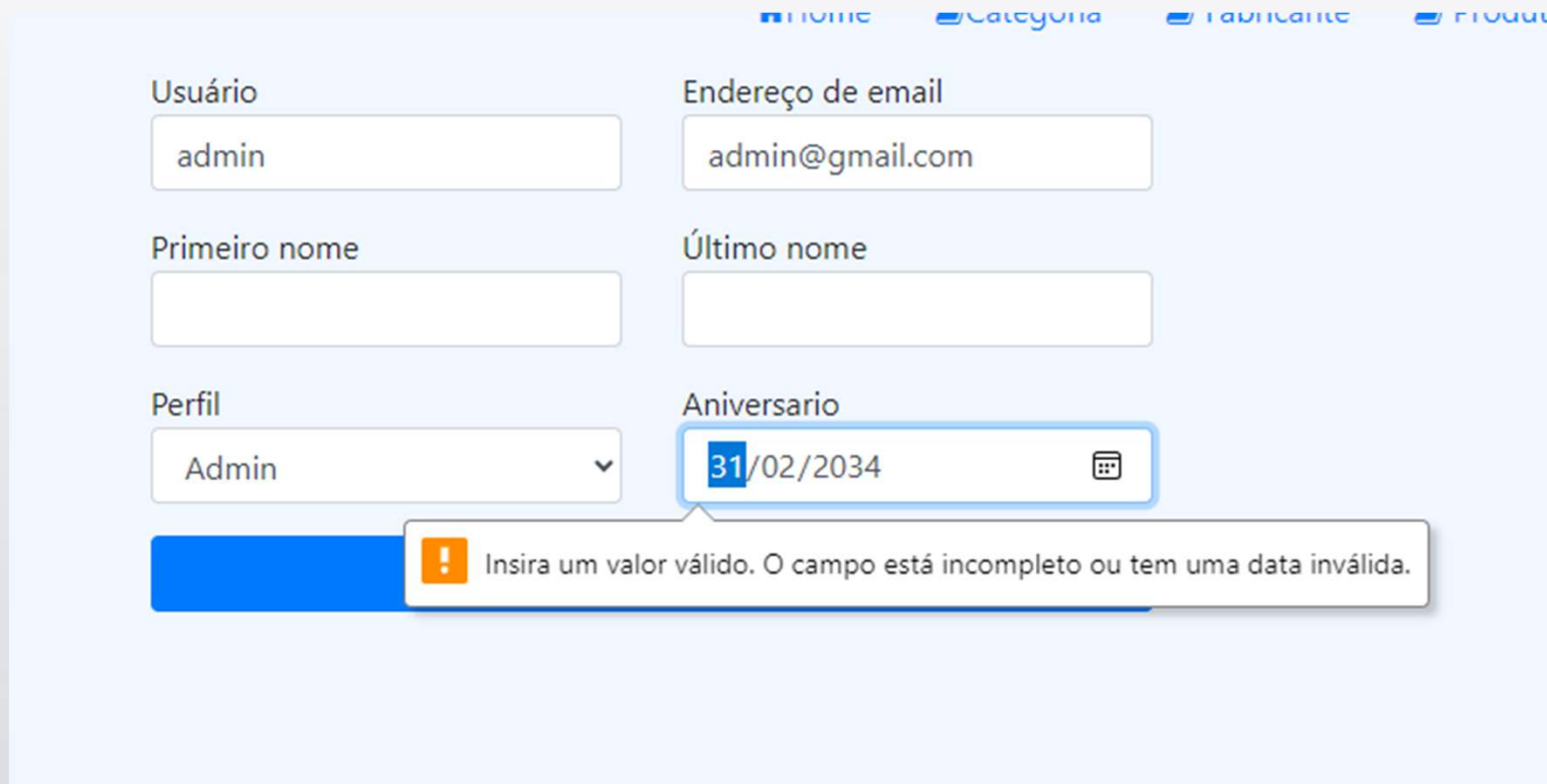
Código do loja/templates/usuario/**usuario-edit.html**:

```
loja > templates > usuario > <> usuario-edit.html > ...
6  {% block content %}
7  <div id="content">
8    <div class="container-fluid">
9      <div class="row">
10        <form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4" method="POST" action="">
11          <div class="row">
12            {% csrf_token %}
13            {% for f in userForm %}
14              {% if not f.is_hidden %}
15                <div class="form-group col-md-6">
16                  {{ f.label }}
17                  {{ f }}
18                  <!-- Adicione as linhas a seguir -->
19                  {% for error in f.errors %}
20                    <div class="invalid-feedback" style="display: initial;">{{ error }}</div>
21                  {% endfor %}
22                  <!-- Até aqui -->
23                </div>
24              {% else %}
25                {{ f }}
26              {% endif %}
27            {% endfor %}
28            {% for f in usuarioForm %}
29              {% if not f.is_hidden %}
30                <div class="form-group col-md-6">
31                  {{ f.label }}
32                  {{ f }}
33                  <!-- Adicione a linha a seguir -->
34                  {% for error in f.errors %}
35                    <div class="invalid-feedback" style="display: initial;">{{ error }}</div>
36                  {% endfor %}
37                  <!-- Até aqui -->
```

Django

Capítulo 9 - Trabalhando com Forms

Rode a aplicação e, para testar os campos de erro, tente gravar uma data inválida



The screenshot shows a Django form with the following fields:

- Usuário**: Text input with value "admin".
- Endereço de email**: Text input with value "admin@gmail.com".
- Primeiro nome**: Empty text input.
- Último nome**: Empty text input.
- Perfil**: Dropdown menu with value "Admin".
- Aniversario**: Date input with value "31/02/2034".

A validation error message is displayed at the bottom:

! Insira um valor válido. O campo está incompleto ou tem uma data inválida.

Django

Capítulo 9 - Trabalhando com Forms

Para finalizar a interface vamos adicionar uma mensagem de "dados salvos" ou de "erro ao salvar" no formulário. Altere o arquivo de visão, **loja/views/UsuarioView.py**, para incluir a mensagem conforme apresentado a seguir.

```
from django.shortcuts import render, redirect, get_object_or_404
<!--Resto do código-->
def edit_usuario_view(request):
    usuario = get_object_or_404(Usuario, user=request.user)
    emailUnused = True
    # Adicione a linha a seguir
    message = None
    <!--Resto do código-->
    if usuarioForm.is_valid() and userForm.is_valid() and emailUnused:
        usuarioForm.save()
        userForm.save()
        # Adicione a linha a seguir
        message = { 'type': 'success', 'text': 'Dados atualizados com sucesso' }
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 2 do `loja/views/UsuarioView.py`

```
# Adicione as linhas a seguir
else:
    # Aqui verificamos se é do tipo post, para que na primeira vez que a página
    # carregar a mensagem não apareça, já que no primeiro carregamento não enviamos um post, o
    # form é dado como inválido e entra aqui.
    if request.method == 'POST':
        if emailUnused:
            # Se o e-mail não está em uso tiver algum dado inválido.
            message = { 'type': 'danger', 'text': 'Dados inválidos' }
        else:
            # Se o e-mail já está em uso por outra pessoa.
            message = { 'type': 'warning', 'text': 'E-mail já usado' }
    # Até aqui
    # Adicione a chave message a seguir
    context = {
```

Django

Capítulo 9 - Trabalhando com Forms

Parte 3 do **loja/views/UsuarioView.py**

```
'usuarioForm': usuarioForm,  
'userForm': userForm,  
'message': message  
}  
return render(request, template_name='usuario/usuario-edit.html',  
context=context, status=200)
```


Django

Capítulo 9 - Trabalhando com Forms

Imagem da função edit do **loja/views/UsuarioView.py**

```
loja > views > UsuarioView.py > ...
13 def edit_usuario_view(request):
14     print(request.user)
15     usuario = get_object_or_404(Usuario, user=request.user)
16     emailUnused = True
17     # Adicione a linha a seguir
18     message = None
19     if request.method == 'POST':
20         usuarioForm = UserUsuarioForm(request.POST, instance=usuario)
21         userForm = UserForm(request.POST, instance=request.user)
22         verifyEmail = Usuario.objects.filter(user__email=request.POST['email']).exclude(user__id=request.user.id).first()
23         emailUnused = verifyEmail is None
24     else:
25         usuarioForm = UserUsuarioForm(instance=usuario)
26         userForm = UserForm(instance=request.user)
27     if usuarioForm.is_valid() and userForm.is_valid() and emailUnused:
28         usuarioForm.save()
29         userForm.save()
30     # Adicione a linha a seguir
31     message = { 'type': 'success', 'text': 'Dados atualizados com sucesso' }
32     # Adicione as linhas a seguir
33     else:
34         # Aqui verificamos se é do tipo post, para que na primeira vez que a página carregar a mensagem não apareça, já que no primeiro
35         # carregamento não enviamos um post, o form é dado como inválido e entra aqui.
36         if request.method == 'POST':
37             if emailUnused:
38                 # Se o e-mail não está em uso tiver algum dado inválido.
39                 message = { 'type': 'danger', 'text': 'Dados inválidos' }
40             else:
41                 # Se o e-mail já está em uso por outra pessoa.
42                 message = { 'type': 'warning', 'text': 'E-mail já usado' }
43     # Até aqui
44     # Adicione a chave message a seguir
45     usuarioForm = UserUsuarioForm(instance=usuario)
46     userForm = UserForm(instance=request.user)
47     context = {
48         'usuarioForm': usuarioForm,
49         'userForm': userForm,
50         'message': message
51     }
```

Ativar o Win
Acesse Configu

Django

Capítulo 9 - Trabalhando com Forms

As alterações feitas vão definir uma mensagem de sucesso nos seguintes cenários:

- Caso o formulário seja válido

- Caso estejamos carregando a tela pela primeira vez a mensagem será do tipo `None` e não exibirá nada

- Caso tentemos submeter o formulário e haja algum dado inválido, ele informará que os dados enviados são inválidos ou que o usuário está tentando salvar um e-mail que já existe na plataforma.

É necessário adicionar o trecho de código ao html para apresentar a mensagem. Esse trecho está no slide seguinte.

Django

Capítulo 9 - Trabalhando com Forms

Abra o arquivo loja/templates/usuario/**usuario-edit.html** e faça as alterações apresentadas a seguir:

```
{% extends "base.html" %}
<!--Resto do código-->
<form class="col-md-6 col-lg-4 offset-md-3 offset-lg-4" method="POST"
  action="">
  <!-- Adicione as linhas a seguir -->
  {% if message is not None %}
  <div class="alert alert-{{ message.type }}">{{ message.text }}</div>
  {% endif %}
  <!-- Até aqui -->
  <div class="row">
    {% csrf_token %}
  <!--Resto do código-->
```

Django

Capítulo 9 - Trabalhando com Forms

Agora teste a mensagem ajustando o e-mail para outro já utilizado, é necessário adicionar um email no(s) usuário(s) comuns que criamos. Basta ir na tabelas de usuários nativa do Django e clicar no usuário em que se deseja adicionar o email e salvar a alteração, em seguida tente colocar esse email no nosso formulário de edição de usuário.

Modificar usuario

usuario1

Usuário:
Obrigatório. 150 caracteres ou menos. Letras, números e @.

Senha: **algoritmo:** pbkdf2_sha256 **iterações:** 600000 **salt:**
Senhas brutas não são armazenadas, então não há como vis

Informações pessoais

Primeiro nome:

Último nome:

Endereço de email:

E-mail já usado

Usuário

Endereço de email

Primeiro nome

Último nome

Perfil

Aniversario

Salvar